~minsheet~

This is the last lecture with me
→ projects!

Today we are going to do two things

- add functions to our lang, I can't leave you without functions!
- explore different evaluation orders

Functions

I hope I don't need to motivate functions.

In our language, we won't name functions, so it will be equivalent to anonymous terms / lambda expressions in Haskell:

$$(\lambda x.x) :: a \to a$$

Syntax:

types $\quad \tau ::= \cdots \mid \tau_1 \to \tau_2$ function type

pre-terms $\quad e ::= \cdots \mid \lambda x{:}\tau.e$ abstraction $\mid e_1(e_2)$ application

(explicit typing)

Statics:

These are very standard, you will see them time and time again in the PL literature:

$$\text{LAM}\ \frac{\Gamma, x:\sigma \vdash e:\tau}{\Gamma \vdash \lambda x{:}\sigma.e : \sigma \to \tau}$$

Lam says that the lambda abstraction $\lambda x{:}\sigma.e$ expands the context with $x:\sigma$, allowing the body to be typed with τ.

$$\text{APP}\ \frac{\Gamma \vdash e_1 : \sigma \to \tau \quad \Gamma \vdash e_2 : \sigma}{\Gamma \vdash e_1(e_2) : \tau}$$

Application says that if e_1 is a function and e_2 has the correct argument type, then the application of the function e_1 to the arg e_2 will have the result type of the function

Q How would we write a function for doubling a number in the STLC?

$$\lambda z{:}\text{Num}.\text{plus}(z\,z)$$

For the dynamics, we want to make lambda expressions inactive, so we only perform the application when the function is fully reduced.

Q How should we specify this in dynamics

$$\text{VAL-LAM}\ \frac{}{\lambda x{:}\tau.e\ \text{val}}$$

$$\text{D-APP-1}\ \frac{e_1 \mapsto e_1'}{e_1(e_2) \mapsto e_1'(e_2)}$$

$$\text{D-BETA}\ \frac{}{(\lambda x{:}\tau.e_1)(e_2) \mapsto e_1[e_2/x]}$$

Note that it is standard to call the act of applying the function (aka subbing in its (first) argument) beta-reduction ($\to_\beta$)

Of course, this use of substitution necessitates us expanding the definition of subst in the meta lang to cover the newest lambda expressions and applications.

$$(e(e_2))[e/x] \stackrel{\text{def}}{=} (e_1[e/x](e_2[e/x]))$$

Again we just propagate the application of the subst into the sub-expressions

$$(\lambda y{:}\tau.u)[e/x] \stackrel{\text{def}}{=} (\lambda y{:}\tau.u[e/x])$$

Again, we have been careful to employ the Barendregt convention + named carefully

Adding functions to our little language gives us the constants + functions fragment of the Simply Typed Lambda Calc

Our lang = constants + functions fragment of the Simply-Typed λ-calc.

The fun thing about the way we have defined functions, is that it supports higher-order functions

This specification of functions supports higher-order (HO) functions

HO functions = functions that take other functions as an argument

e.g.

```
twice :: (a->a) -> a -> a
twice f x = f(f x)
```

—— bonus

Let's play with our function terms!

Q Can someone help me write the function add : Num→Num→Num

(it might be helpful to write it in Haskell, then translate it).

(see lecture note 8 for solution)

(Q let's justify its well-typedness with a derivation)

(Q lets evaluate it)

twice.

Q How would we write twice in STLC?

(Q Justify its well-typedness)

(Q Evaluate it with the dynamics)

Call-by-name Vs. Call-by-value

When I introduced the dynamics of our little language, we discussed how search transitions control order of evaluation. We will now formally discuss different evaluation strategies.

So far in the course, the evaluation strategy that we have used is

call-by-name (CBN)
= where variables represent terms
(as opposed to values as we will come on to)

We can see this evaluation strategy in the substitution lemma, and the dynamics of constructs that involve substitution

Example:

$f(x) = x + x$

$f(1+1)$
$\rightarrow (1+1) + (1+1)$ — Notice how $1+1$ gets subbed in whole-sale as a term.
$\rightarrow 2 + (1+1)$
$\rightarrow 2 + 2$
$\rightarrow 4$

There is another approach we could of taken, which for this example would reduce the number of evaluation steps.

Introducing

call-by-value
= where variables represent values

i.e. arguments to functions (or any terms that get bound) must be reduced to values before they get substituted in

Example:

$f(1+1)$
$\rightarrow f(2)$
$\rightarrow 2 + 2$
$\rightarrow 4$

3 eval steps vs. 4.

A key thing to highlight is that

Evaluation order does not effect pure results.

Note that our example evaluated to 4 in both cases, the only difference is the number of steps to get there.

But assuming that when we run programs we cannot note the difference between eval steps

The difference is only noticable in the presence of effects

Q What does it mean to you when I say effects?

Q who can name some effects?

Example effects: IO, state, failure, non-det

Q Pick one of these, and explain why it might make eval order noticable

In Crishna's half, you will be working with an effectful language featuring mutable state, but for now, back to CBV.

You've actually met loads of CBV languages in your degree, because almost all languages with effects are CBV

CBV languages:
- C
- Java
- Scala
- JS
- OCaml
- Scheme
- ...

CBN is very strongly related to lazy eval of purely functional languages

Haskell = call-by-need
↑
optimised variant of CBN.

Call-by-value λ-calc

Let's now explore how the CBV version of our language differs from what we currently have

The key change will be that any time we perform a substitution, we only substitute in a value for a variable.

Reference: Simply-typed λ-calculus

Samantha Frohlich

Q Any changes to statics?
No, eval order is dynamics

Figure 1: Statics of the constants-and-functions fragment of the simply-typed λ-calculus

$$\textsc{Var}\ \frac{}{\Gamma, x : \sigma \vdash x : \sigma\ s} \qquad \textsc{Num}\ \frac{n \in \mathbb{N}}{\Gamma \vdash \mathsf{num}[n] : \mathsf{Num}} \qquad \textsc{Plus}\ \frac{\Gamma \vdash e_1 : \mathsf{Num} \qquad \Gamma \vdash e_2 : \mathsf{Num}}{\Gamma \vdash \mathsf{plus}(e_1; e_2) : \mathsf{Num}}$$

$$\textsc{Times}\ \frac{\Gamma \vdash e_1 : \mathsf{Num} \qquad \Gamma \vdash e_2 : \mathsf{Num}}{\Gamma \vdash \mathsf{times}(e_1; e_2) : \mathsf{Num}} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \sigma_1 \qquad \Gamma, x : \sigma_1 \vdash e_2 : \sigma_2}{\Gamma \vdash \mathsf{let}(e_1; x.\, e_2) : \sigma_2}$$

$$\textsc{Lam}\ \frac{\Gamma, x : \sigma \vdash e : \tau}{\Gamma \vdash \lambda x : \sigma.\, e : \sigma \to \tau} \qquad \textsc{App}\ \frac{\Gamma \vdash e_1 : \sigma \to \tau \qquad \Gamma \vdash e_2 : \sigma}{\Gamma \vdash e_1(e_2) : \tau}$$

Q What needs changed in the dynamics?
Rules that perform a step

Figure 2: Dynamics of the constants-and-functions fragment of the simply-typed λ-calculus

$$\textsc{Val-Lam}\ \frac{}{\lambda x : \tau.\, e\ \mathsf{val}} \qquad \textsc{Val-Num}\ \frac{n \in \mathbb{N}}{\mathsf{num}[n]\ \mathsf{val}} \qquad \textsc{D-Plus}\ \frac{n_1 + n_2 = n}{\mathsf{plus}(\mathsf{num}[n_1]; \mathsf{num}[n_2]) \longmapsto \mathsf{num}[n]}$$

$$\textsc{D-Plus-1}\ \frac{e_1 \longmapsto e_1'}{\mathsf{plus}(e_1; e_2) \longmapsto \mathsf{plus}(e_1'; e_2)} \qquad \textsc{D-Plus-2}\ \frac{e_1\ \mathsf{val} \qquad e_2 \longmapsto e_2'}{\mathsf{plus}(e_1; e_2) \longmapsto \mathsf{plus}(e_1; e_2')}$$

$$\textsc{D-Let}\ \frac{e_1\ \mathsf{val}}{\mathsf{let}(e_1; x.\, e_2) \longmapsto e_2[e_1/x]} \qquad \textsc{D-App-1}\ \frac{e_1 \longmapsto e_1'}{e_1(e_2) \longmapsto e_1'(e_2)} \qquad \textsc{D-Beta}\ \frac{e_2\ \mathsf{val}}{(\lambda x : \tau.\, e_1)(e_2) \longmapsto e_1[e_2/x]}$$

D-LET + D-BETA need what they sub to be constrained to a val.

AND we need extra rules to reduce non-values to values:

$$\textsc{D-App-2}\ \frac{e_1\ \mathsf{val} \qquad e_2 \longmapsto e_2'}{e_1(e_2) \longmapsto e_1(e_2')}$$

$$\textsc{D-Let-1}\ \frac{e_1 \longmapsto e_1'}{\mathsf{let}(e_1; x.\, e_2) \longmapsto \mathsf{let}(e_1'; x.\, e_2)}$$

Call-by-value λ-calc-cont

Progress + Preservation hold for CBV-STLC.

(the sheet will explore this)